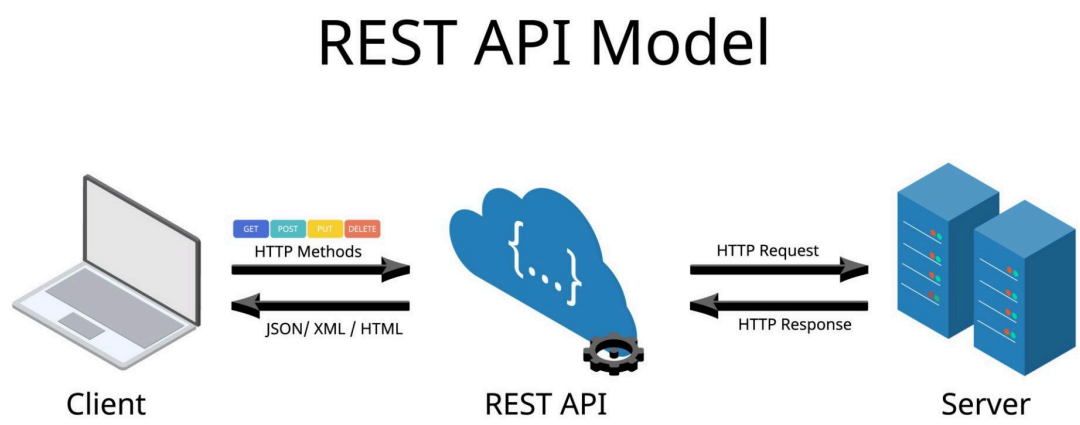
2.REST – Representational State Transfer

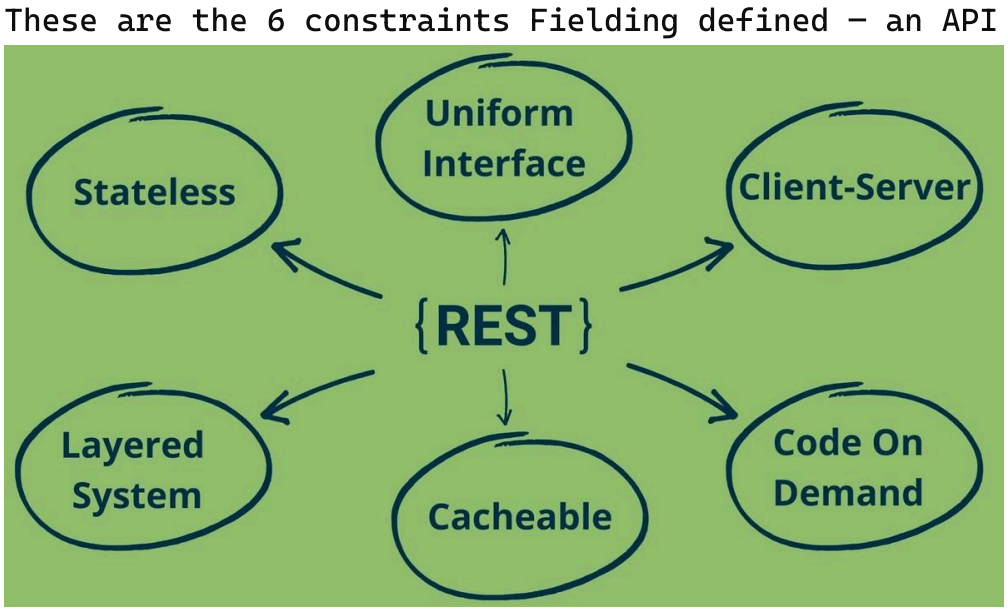
What is REST

- REST is an architectural style for designing networked applications
- It is not a protocol, not a framework, not a library – it is a set of constraints and principles
- Coined by Roy Fielding in his 2000 PhD dissertation
- Any API that follows these constraints is called a RESTful API
- REST uses HTTP as its communication protocol – it leverages everything HTTP already provides
- The core idea – treat everything on the server as a resource, and interact with those resources using standard HTTP methods



REST Constraints and Principles

- These are the 6 constraints Fielding defined – an API must follow these to be truly RESTful



1. Stateless

- Every request from client to server must contain all information needed to process it
- Server stores no session state about the client between requests
- Authentication token, context, parameters – all must be in the request itself
- Why it matters for system design – any server instance can handle any request, enabling horizontal scaling
- If state lived on the server, you need sticky sessions – that kills scalability

2. Client-Server

- Client and server are separated and evolve independently
- Client handles the UI and user experience
- Server handles data storage, business logic, and processing
- They communicate only through the API contract
- Why it matters – you can rewrite the entire frontend without touching the backend and vice versa, as long as the API contract stays the same

3. Cacheable

- Responses must explicitly declare whether they can be cached or not
- If a response is cacheable, clients and intermediaries (CDNs, proxies) can reuse it

- Done through Cache-Control, ETag, Last-Modified headers
- Why it matters — reduces server load dramatically, improves latency, enables CDN scaling

4. Uniform Interface

- This is the most important constraint — it is what makes REST "REST"
- Standardised way of interacting with all resources regardless of the service
- Has 4 sub-constraints:
  - Resource identification — every resource is identified by a URL (URI)
  - Manipulation through representations — client interacts with resources through representations (JSON, XML), not the resource directly
  - Self-descriptive messages — each message contains enough information to describe how to process it (Content-Type header, HTTP method)
  - HATEOAS — responses include links to related actions (covered separately below)
- Why it matters — any client can interact with any REST API the same way, no custom protocol needed per service

5. Layered System

- Client does not know if it is talking directly to the server or through intermediaries
- Intermediaries can be load balancers, reverse proxies, CDNs, API gateways
- Each layer only knows about the layer immediately next to it
- Why it matters — you can add caching layers, security layers, load balancing layers without changing the client or the backend server

6. Code on Demand (Optional)

- Server can optionally send executable code to the client
- Example — server sends JavaScript that the browser executes
- This is the only optional constraint
- Rarely discussed in system design interviews — know it exists, do not go deeper

---

Resource Naming Conventions

- URLs in REST represent resources — the naming of these URLs is critical to a clean API design

Nouns not verbs

- Resources are things, not actions
- The HTTP method already describes the action — the URL should describe the thing
- Wrong — /getUser, /createProduct, /deleteOrder
- Right — /users, /products, /orders
- The verb is already in the HTTP method: GET /users, POST /products, DELETE /orders
- Using verbs in URLs is a sign of RPC-style thinking, not REST thinking

Plural vs Singular

- Use plural nouns for collections — it is the widely accepted standard
- /users — represents the collection of all users
- /users/123 — represents a specific user with ID 123
- /products — collection of products
- /products/456 — specific product
- Consistency matters more than the rule itself — pick plural and stick to it everywhere

Nested resources

- When a resource exists in the context of another resource, nest the URL
- /users/123/orders — all orders belonging to user 123
- /users/123/orders/789 — specific order 789 belonging to user 123
- /posts/456/comments — all comments on post 456
- Rule of thumb — do not nest more than 2 levels deep
  - Deeper than that becomes hard to read and maintain
  - /users/123/orders/789/items/1 — borderline acceptable

- /users/123/orders/789/items/1/reviews/2 — too deep, flatten this
- Flatten deep nesting: /reviews/2 with a query param or body reference to the parent

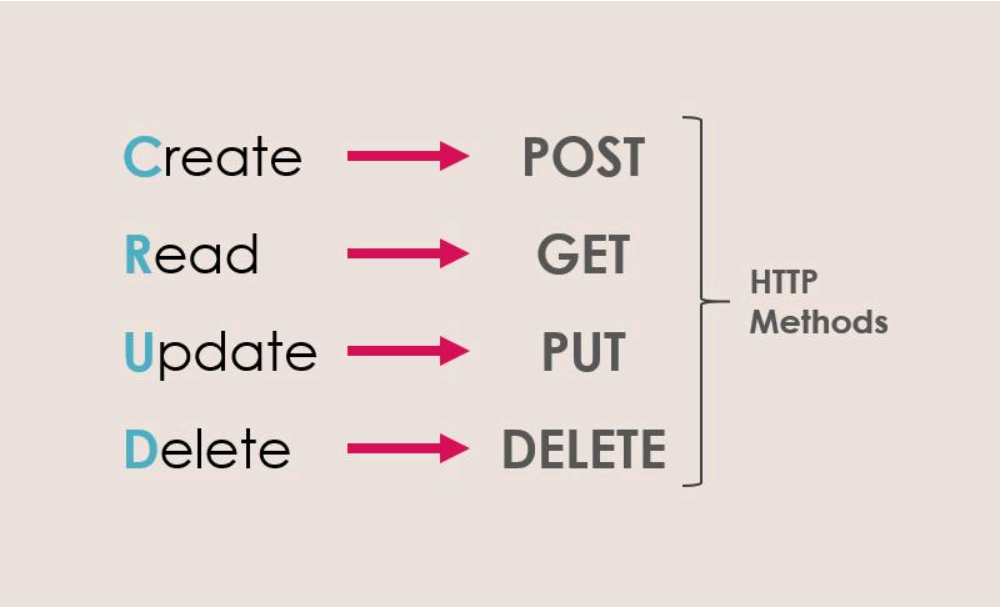
URL structure best practices

- Always lowercase — /users not /Users
- Use hyphens for readability, not underscores — /product-categories not /product_categories
- No trailing slash — /users not /users/
- No file extensions — /users not /users.json (use Accept header for format negotiation)
- Hierarchical structure should reflect relationships
- Keep URLs stable — once published, changing URLs breaks clients

---

CRUD Mapping to HTTP Methods

- REST maps the four basic data operations to HTTP methods



GET → Read

- Retrieve a resource or collection of resources
- GET /users — fetch all users
- GET /users/123 — fetch user with ID 123
- No body in request
- Safe and idempotent — never modifies data, same result every time
- Response is cacheable

POST → Create

- Create a new resource
- POST /users — create a new user, body contains user data
- Server assigns the ID, client does not specify it
- Not idempotent — calling POST twice creates two records
- Returns 201 Created with the new resource (or its URL) in the response

PUT → Update (Full)

- Replace an entire resource with what is sent in the request
- PUT /users/123 — replace user 123 completely with the body sent
- If a field is missing from the body, it is removed or set to null
- Idempotent — calling PUT multiple times with the same data produces the same result
- Client must send the complete resource representation

PATCH → Update (Partial)

- Update only the fields specified in the request
- PATCH /users/123 — only update the fields included in the body
- If body contains only { "email": "new@email.com" }, only the email changes
- More efficient than PUT when changing one field in a large object
- Idempotent in practice though technically debated

DELETE → Delete

- Remove a resource

- DELETE /users/123 — delete user with ID 123
- No body typically
- Idempotent — deleting something already deleted should return 204 or 404, not crash

---

Idempotency in REST

What idempotency means

- An operation is idempotent if performing it multiple times has the same effect as performing it once
- The state of the server after 1 call and after 10 calls is identical
- Not about the response being identical — about the server state being identical

Which methods are idempotent

- GET — idempotent, read-only, no state change
- PUT — idempotent, replaces resource with same data every time
- DELETE — idempotent, resource is gone after first call, subsequent calls change nothing
- PATCH — generally idempotent in practice
- POST — not idempotent, each call creates a new resource
- POST with idempotency key — you can make POST idempotent by sending a unique key
  - Client sends Idempotency-Key: unique-uuid-here in the header
  - Server stores this key and the result
  - If same key is sent again, server returns the stored result instead of processing again
  - Stripe, PayPal use this for payment APIs — prevents duplicate charges on retry

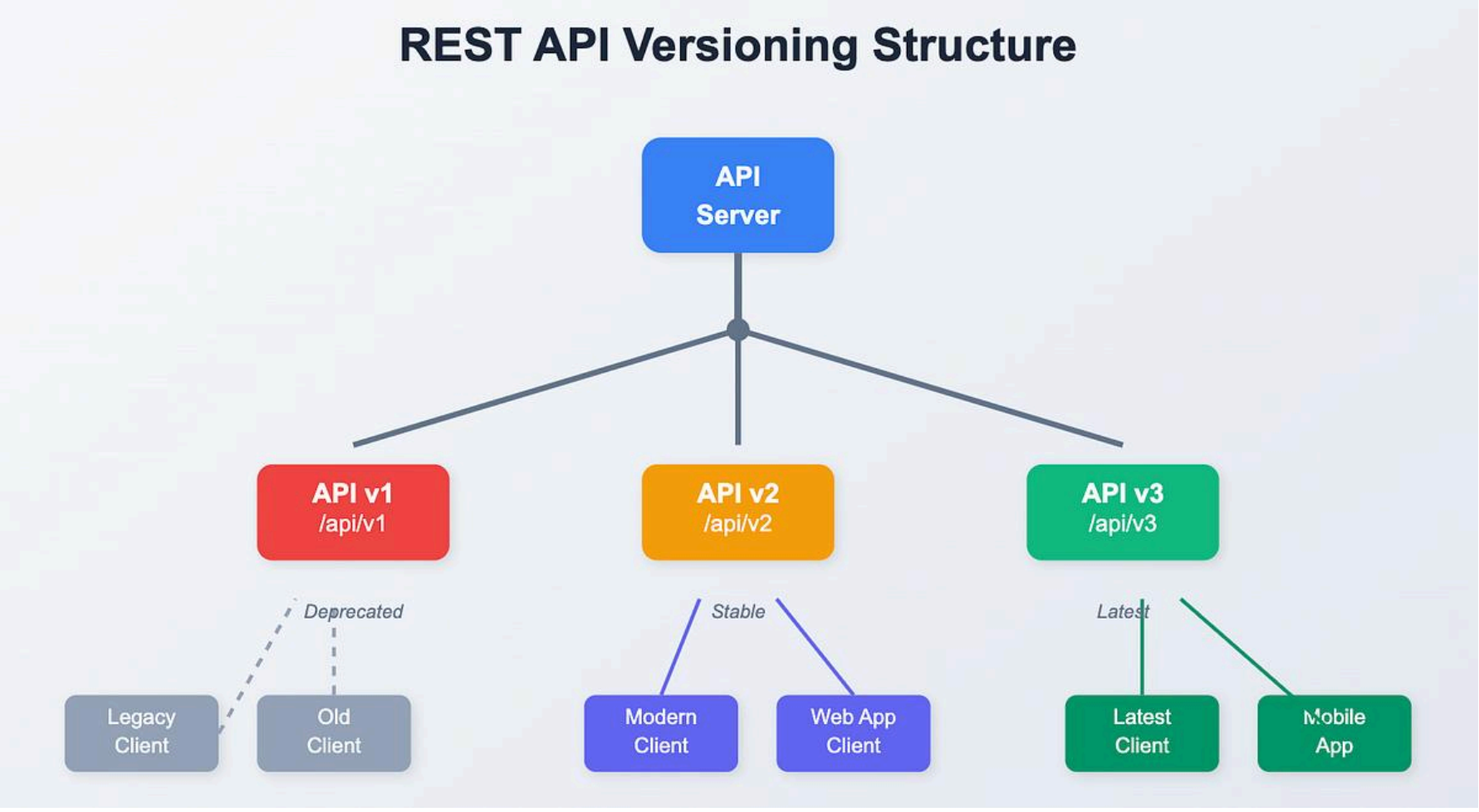
Why idempotency matters in system design

- Networks are unreliable — requests time out, connections drop
- Client does not know if the server processed the request or not before the timeout
- Safe retry strategy — if the operation is idempotent, just retry it
- If not idempotent (POST), retrying blindly creates duplicate data
- In distributed systems, at-least-once delivery is common — messages may be processed more than once
- Idempotent operations make this safe — processing the same message twice has no side effect
- This is critical in payment systems, order systems, any system where duplicate operations cause real-world harm

---

REST API Versioning

- APIs evolve — you add fields, change behaviour, remove deprecated features
- External consumers depend on stability — breaking changes break their applications
- Versioning lets you evolve the API while keeping old versions running for existing consumers



URL versioning

- Version is embedded in the URL path
- /v1/users, /v2/users
- Most widely used approach
- Pros:
  - Visible and explicit — easy to see which version you are using
  - Easy to test in a browser or curl
  - Easy to route at the API gateway level — /v1 goes to old service, /v2 goes to new service
  - Easy to document separately
- Cons:
  - URL should identify a resource, not a version — purists argue this violates REST principles
  - Changing version means updating every URL in client code
- When to use — public APIs, third party developer APIs, when visibility and simplicity matter

Header versioning

- Version is sent in a custom request header
- API-Version: 2 or Accept: application/vnd.myapi.v2+json
- Pros:
  - Clean URLs — the resource URL does not change
  - More RESTful in the purist sense
- Cons:
  - Not visible in the URL — harder to test, harder to debug
  - Cannot be tested directly in a browser
  - Harder to cache — caching is URL-based, header-versioned responses can be tricky
- When to use — internal APIs where clients are controlled and you want clean URLs

Query param versioning

- Version is passed as a query parameter
- /users?version=2 or /users?api_version=2
- Pros:
  - Optional — clients that do not send the param get a default version
  - Easy to add without changing URL structure
- Cons:
  - Query params are for filtering and searching, not versioning — semantically wrong
  - Less clean, easy to forget
  - Caching complications — caches must treat different query param values as different resources
- When to use — rarely preferred, sometimes used for quick internal versioning

Which to use when

- Public API with external developers → URL versioning, it is the clearest and most widely understood
- Internal microservices API → header versioning keeps URLs clean and you control all consumers
- Quick prototype or internal tool → query param versioning for simplicity
- In FAANG interviews → default to URL versioning unless you have a specific reason to choose otherwise, and explain the tradeoff

---

Pagination

Why pagination exists

- A database table can have millions of records
- Returning all of them in one response would:
  - Consume enormous memory on the server building the response
  - Transfer massive amounts of data over the network
  - Crash or overwhelm the client trying to render it
- Pagination breaks results into smaller chunks
- Client fetches one page at a time

Offset Based Pagination

How it works

- Client specifies two parameters — offset (how many records to skip) and limit (how many to return)
- GET /products?offset=0&limit=20 — first 20 products
- GET /products?offset=20&limit=20 — next 20 products
- GET /products?offset=40&limit=20 — next 20 products
- The database query uses OFFSET and LIMIT: SELECT * FROM products LIMIT 20 OFFSET 40

Pros

- Simple to implement and understand
- Client can jump to any page directly — want page 50? offset=980&limit=20
- Easy to display page numbers in UI

Cons and performance problems at scale

- The OFFSET problem — the database does not skip rows efficiently
  - With OFFSET 1000000, the database reads and discards 1 million rows before returning the 20 you want
  - Gets slower as offset increases — O(offset) time complexity
  - On a table with 100 million rows, OFFSET 99999980 is catastrophically slow
- The consistency problem — if new records are inserted while a user is paginating:
  - User fetches offset=0, gets products 1–20
  - A new product is inserted at the top
  - User fetches offset=20, gets products 21–40 — but product 20 from the first page is now at position 21 and appears again
  - Records shift, causing duplicates or skipped items
- Not suitable for real-time or frequently updated datasets

Cursor Based Pagination

How it works

- Instead of offset, the server returns a cursor — an opaque pointer to a position in the dataset
- First request: GET /posts?limit=20 — server returns 20 posts and a cursor

- Response: { data: [...], next_cursor: "eyJpZCI6MjB9" }
- Next request: GET /posts?limit=20&cursor=eyJpZCI6MjB9
- Server decodes the cursor to find the position and fetches the next 20 from there
- The cursor typically encodes the ID or timestamp of the last item returned
- Database query: SELECT * FROM posts WHERE id > last_id LIMIT 20
- This uses an indexed column — extremely fast regardless of position

Why it solves offset problems

- No scanning and discarding rows — WHERE id > last_id uses the index directly
- O(1) position lookup instead of O(offset)
- Consistent — new inserts do not shift positions because you are paginating by ID, not position number
- If a new post is inserted, it gets a new ID larger than the current cursor — does not affect your current page

Use cases

- Twitter feed, Facebook timeline, Instagram feed — endless scroll interfaces
- Any real-time or frequently updated dataset
- Large datasets where offset performance would degrade
- Anywhere you do not need random page access

Tradeoffs of cursor pagination

- Cannot jump to arbitrary pages — you must paginate sequentially
- No page numbers — cannot say "go to page 50"
- Cursor is opaque — client cannot construct a cursor, only use what server provides

---

Page Based Pagination

- A variant of offset pagination where client specifies page number instead of raw offset
- GET /products?page=3&per_page=20
- Server converts: offset = (page - 1) * per_page
- Same underlying mechanics and same problems as offset pagination
- Just a friendlier interface for the client
- Common in admin dashboards, CMS systems, anywhere random page access is needed

---

Keyset Pagination

- Very similar to cursor pagination — uses the value of an indexed column as the anchor
- GET /orders?after_id=12345&limit=20
- Query: SELECT * FROM orders WHERE id > 12345 ORDER BY id LIMIT 20
- Difference from cursor — the key is visible and meaningful (an actual ID), not an encoded opaque cursor
- Pros — fast, consistent, simple to implement
- Cons — the key must be unique and ordered, works best with auto-increment IDs or timestamps
- In practice, cursor pagination and keyset pagination are often used interchangeably in conversation

---

Filtering and Sorting

Query params for filtering

- Filtering narrows down the result set based on conditions
- Standard pattern — use query parameters for each filter
- GET /products?category=electronics — filter by category
- GET /products?category=electronics&brand=sony — multiple filters combined with AND
- GET /products?min_price=100&max_price=500 — range filter

- GET /users?status=active&role=admin — filter by multiple fields
- For complex filters, some APIs use a filter query param with structured syntax:
  - GET /products?filter[category]=electronics&filter[price][gte]=100
- Keep filter param names intuitive and consistent across all endpoints

Sorting patterns

- Use a sort query parameter
- GET /products?sort=price — sort by price ascending
- GET /products?sort=-price — prefix with minus for descending (common convention)
- GET /products?sort=price&order=desc — explicit order parameter (alternative convention)
- Multiple sort fields: GET /products?sort=-created_at,name — sort by date descending, then name ascending
- Always define a default sort order — do not return random order when no sort is specified

Combining filter and sort

- All query params combine naturally
- GET /products?category=electronics&min_price=100&sort=-created_at&page=2&per_page=20
- This reads as: give me electronics products priced above 100, sorted by newest first, page 2, 20 per page
- The server must apply filters first, then sort, then paginate — in that order
- Applying pagination before filtering would return wrong results

---

Error Response Design

Consistent error format

- Every error response across your entire API must follow the same structure
- Nothing is more frustrating for API consumers than different error formats on different endpoints
- Define one error schema and use it everywhere
- A good error response structure:

```
{
  "error": {
    "code": "INVALID_EMAIL",
    "message": "The email address provided is not valid",
    "details": "email must be a valid RFC 5322 address",
    "request_id": "req_8f3bc9d2",
    "documentation_url": "https://docs.example.com/errors/INVALID_EMAIL"
  }
}
```

Error codes vs HTTP status codes

- HTTP status codes are coarse — 400 just means bad request, but why?
- Application error codes give precise machine-readable reason
- HTTP status code tells the client the category of error
- Application error code tells the client exactly what went wrong
- Client code can switch on the application error code to handle specific cases
- Example:
  - HTTP 400 + code INVALID_EMAIL → show email validation error
  - HTTP 400 + code MISSING_REQUIRED_FIELD → highlight the missing field
  - HTTP 400 + code DUPLICATE_USERNAME → suggest a different username
  - All are 400 but each requires different client handling

Error messages for developers

- Technical, precise, actionable
- Should tell the developer exactly what is wrong and how to fix it
- Include field names, constraints violated, expected formats

- Example: "Field 'date_of_birth' must be in ISO 8601 format (YYYY-MM-DD), received '15-04-1990'"
- Include a request_id — developer can send this to support for debugging

Error messages for users

- Human friendly, non-technical, empathetic
- Never expose internal details — stack traces, database errors, server paths
- Keep it simple: "We could not process your request. Please check your email address and try again."
- The developer message and user message are different fields — client decides which to show where

---

HATEOAS

What it is

- HATEOAS stands for Hypermedia As The Engine Of Application State
- It is the final and most advanced REST constraint
- The idea — API responses should include links to related actions and resources
- The client should be able to navigate the entire API by following links, without hardcoding URLs
- Named after how the web works — you visit a page, it has links, you follow them, you discover more pages

Why it exists

- In a typical API, the client hardcodes URLs — it knows /users/123/orders exists
- If the server changes the URL structure, the client breaks
- With HATEOAS, the server tells the client what actions are available and where
- Client just follows the links the server provides — never constructs URLs itself
- Client and server are more loosely coupled

What it looks like

- A response for GET /users/123 with HATEOAS:

```
{
  "id": 123,
  "name": "Arjun",
  "email": "arjun@example.com",
  "_links": {
    "self": { "href": "/users/123" },
    "orders": { "href": "/users/123/orders" },
    "update": { "href": "/users/123", "method": "PUT" },
    "delete": { "href": "/users/123", "method": "DELETE" }
  }
}
```

- Client reads the links from the response instead of constructing them

Practical use cases

- Payment APIs — response includes links to confirm, cancel, or refund a payment
- E-commerce — order response includes links to track, cancel, or return
- GitHub API uses HATEOAS — responses include links to related resources
- Useful when the available actions depend on state — a cancelled order shows different links than a pending order

Reality check

- HATEOAS is rarely fully implemented in practice
- Most production REST APIs skip it
- Most clients hardcode URLs anyway

- Know what it is, know why it exists, but do not be surprised when real APIs do not implement it

---

REST Best Practices

- Use nouns in URLs, not verbs
- Use plural resource names consistently
- Version your API from day one — adding versioning later is painful
- Always paginate collection endpoints — never return unbounded lists
- Use consistent error response format across all endpoints
- Use proper HTTP status codes — do not return 200 for errors
- Make GET, PUT, DELETE idempotent — document which operations are not
- Use HTTPS always — never expose a REST API over plain HTTP
- Implement rate limiting — protect your backend from abusive clients
- Use query params for filtering, sorting, pagination — not path params
- Return the created/updated resource in POST and PUT responses — saves the client an extra GET
- Support partial responses with field selection — GET /users/123?fields=id,name,email reduces payload size
- Document your API — no documentation means no adoption
- Use consistent date formats — always ISO 8601 (2026-04-20T10:30:00Z)
- Implement health check endpoints — GET /health returns server status, used by load balancers

---

REST Limitations

- These are important for system design — knowing REST's limits tells you when to use something else

Over-fetching

- REST returns entire resource representations
- If you need only the user's name, you still get the entire user object with 20 fields
- Wastes bandwidth, especially on mobile networks
- GraphQL was built specifically to solve this

Under-fetching and N+1 problem

- A single screen may need data from multiple resources
- To show a user's profile with their orders and reviews, you need:
  - GET /users/123
  - GET /users/123/orders
  - GET /users/123/reviews
- Three separate round trips for one screen
- On mobile, each round trip adds latency
- GraphQL solves this with a single query for all needed data

Versioning complexity

- Maintaining multiple API versions simultaneously is expensive
- v1, v2, v3 all running at once means supporting and testing all three
- Old versions must be kept alive for years until all consumers migrate

Not ideal for real-time

- REST is request-response — client must ask, server responds
- For real-time updates (live scores, chat messages, notifications), client must poll repeatedly
- Polling is inefficient — most polls return no new data
- WebSockets or Server-Sent Events are better for real-time use cases

Lack of strict contract

- REST has no formal schema by default

- Client and server must agree on field names, types, and structure informally
- If the server changes a field name, clients break silently
- gRPC with Protocol Buffers solves this with a strict, typed schema

Chatty in microservices

- In microservices, services make many REST calls to each other
- Each call has HTTP overhead — headers, TLS, TCP connection
- gRPC is significantly more efficient for internal service-to-service communication

No built-in streaming

- REST handles discrete request-response cycles
- Streaming large datasets or continuous data requires workarounds
- gRPC has native streaming support — much cleaner for these use cases

---